

Объектно-ориентированное программирование

Задание 1.

Реализовать, с учетом ооп подхода, приложение. Для проведения исследований с генеалогическим древом.

Идея: описать некоторое количество компонент, например - модель человека и дерева. Под “проведением исследования” можно понимать например получение всех детей выбранного человека.

Подсказка № 1

Создайте базовую модель класса "Person" с полями и методами. Начните с создания класса, представляющего человека. Учитывайте, что каждому человеку может соответствовать несколько атрибутов, таких как имя, год рождения, мать, отец и список детей. Это позволит вам создать основные связи внутри древа.

Подсказка № 2

Используйте методы для работы с родительскими связями. Добавьте методы, которые позволяют устанавливать связи между объектами класса "Person". Например, метод `setMother()` или `setFather()` позволит установить родственные отношения, а метод `addChild()` — добавить ребенка.

Подсказка № 3

Класс "FamilyTree" будет отвечать за хранение всех объектов "Person" и предоставление функционала для поиска и обработки данных о людях в древе. Например, метод `findPersonByName()` поможет находить человека по имени, а `getChildren()` — получать список детей для конкретного человека.

Подсказка № 4

Помимо базовых функций, таких как добавление людей и поиск детей, вы можете добавить методы для поиска братьев и сестер, предков или построения целого древа для конкретного человека. Это сделает ваше приложение более функциональным.

Подсказка № 5

После написания основных компонентов вашего приложения, протестируйте его на нескольких примерах. Убедитесь, что методы корректно обрабатывают данные и возвращают ожидаемые результаты. Проверьте все сценарии, такие как отсутствие родителей, поиск по имени, вывод списка детей и т.д.

Эталонное решение:

```
import java.util.ArrayList;
```

```
import java.util.List;

// Класс, представляющий человека
class Person {

    private String name;

    private int birthYear;

    private Person mother;

    private Person father;

    private List<Person> children;

    public Person(String name, int birthYear) {

        this.name = name;

        this.birthYear = birthYear;

        this.children = new ArrayList<>();

    }

    public String getName() {

        return name;

    }

    public int getBirthYear() {

        return birthYear;

    }

    public void setMother(Person mother) {

        this.mother = mother;

    }

}
```

```

    public void setFather(Person father) {
        this.father = father;
    }

    public void addChild(Person child) {
        this.children.add(child);
    }

    public List<Person> getChildren() {
        return children;
    }

    public Person getMother() {
        return mother;
    }

    public Person getFather() {
        return father;
    }
}

// Класс для работы с генеалогическим древом
class FamilyTree {
    private List<Person> people;

    public FamilyTree() {

```

```

        this.people = new ArrayList<>();
    }

    public void addPerson(Person person) {
        this.people.add(person);
    }

    public List<Person> getChildren(Person parent) {
        return parent.getChildren();
    }

    public Person findPersonByName(String name) {
        for (Person person : people) {
            if (person.getName().equals(name)) {
                return person;
            }
        }
        return null;
    }
}

public class Main {

    public static void main(String[] args) {

        FamilyTree familyTree = new FamilyTree();

        // Создаем людей

        Person john = new Person("John", 1950);

```

```
Person mary = new Person("Mary", 1955);

Person susan = new Person("Susan", 1980);

// Устанавливаем родительские связи

susan.setMother(mary);

susan.setFather(john);

john.addChild(susan);

mary.addChild(susan);


// Добавляем людей в древо

familyTree.addPerson(john);

familyTree.addPerson(mary);

familyTree.addPerson(susan);


// Пример получения всех детей Джона

List<Person> johnsChildren = familyTree.getChildren(john);

for (Person child : johnsChildren) {

    System.out.println("John's child: " + child.getName());

}

}
```